EvRAG: Enhanced RAG with Small Model Fine-tuning

Liang Zhang

December 3, 2025

Abstract

This paper presents EvRAG, an intelligent question-answering system based on Retrieval-Augmented Generation (RAG). Through domain-adaptive fine-tuning, hybrid retrieval strategies, and reranking optimization, we demonstrate that a fine-tuned small model (Qwen3-8B with 8B parameters) can outperform general-purpose large models (Qwen3-32B with 32B parameters) on domain-specific tasks. Experimental results on 676 manually annotated test samples show that our system achieves a comprehensive accuracy improvement from 80.92% to 92.08%, representing a 13.8% increase, while reducing the number of parameters by 75%. Additionally, system stability improved by 51% (standard deviation decreased from 0.2065 to 0.1012), and ContextPrecision improved by 34.5%. This achievement demonstrates the critical value of domain-adaptive fine-tuning in engineering deployment and provides practical guidance for cost-effective high-precision RAG systems.

Keywords: Retrieval-Augmented Generation, Domain-Adaptive Fine-tuning, Hybrid Retrieval, Reranking, Small Model Optimization

1 Introduction

1.1 Problem Background

With the rapid development of large language models (LLMs), general-purpose large models such as Qwen3-32B demonstrate excellent performance in general scenarios but face challenges of high deployment costs and significant resource consumption. Particularly in edge devices and resource-constrained environments, mobile devices and embedded systems cannot support the deployment and inference of massive parameter models. Meanwhile, domain-specific tasks require high-precision yet cost-effective solutions.

Retrieval-Augmented Generation (RAG) systems face multiple challenges in knowledge question-answering: insufficient retrieval precision, unstable generation quality, and

difficulty ensuring citation accuracy. Traditional RAG approaches typically employ general-purpose large-parameter models and large-parameter embedding models, directly indexing PDF-parsed content into vector databases without targeted optimization.

1.2 Research Motivation

This research aims to explore the following questions: Can small models surpass large models on specific tasks through domain fine-tuning? How can we achieve high-performance question-answering systems in resource-constrained environments? How can we verify the feasibility of engineering deployment and achieve cost-benefit balance? How can we demonstrate the value of domain-adaptive fine-tuning?

1.3 Main Contributions

The main contributions of this paper include the following. First, we implement a complete RAG system containing hybrid retrieval, reranking, LLM generation, and a carefully designed document processing pipeline. Second, we prove that RAG systems built with fine-tuned models can outperform solutions using general-purpose large-parameter models and large-parameter embedding models with PDF parsing and vector database indexing commonly found in commercial chatbots. Third, we provide a complete training data generation and model fine-tuning pipeline. Fourth, we establish a systematic evaluation framework that quantifies the contribution of each component to system performance.

2 System Design

2.1 Overall Architecture

Our EvRAG system adopts a two-layer architecture design, as illustrated in Figure 1. The system consists of two core components: the retrieval layer and the generation layer.

The retrieval layer employs a hybrid retrieval strategy combining keyword matching and semantic similarity. BM25 retrieval provides sparse retrieval based on keyword matching with topk=5, while Milvus vector retrieval offers semantic retrieval based on the BGE-M3 model with topk=10. The system merges BM25 and Milvus retrieval results with deduplication processing, then uses a fine-tuned BGE-Reranker-v2-m3 to rerank the merged documents with topk=5.

The generation layer is responsible for answer generation and post-processing. It constructs structured prompts from user queries and retrieved contexts, generates answers using a fine-tuned Qwen3-8B model with LoRA support, and performs post-processing including citation extraction, output formatting, and image path parsing.

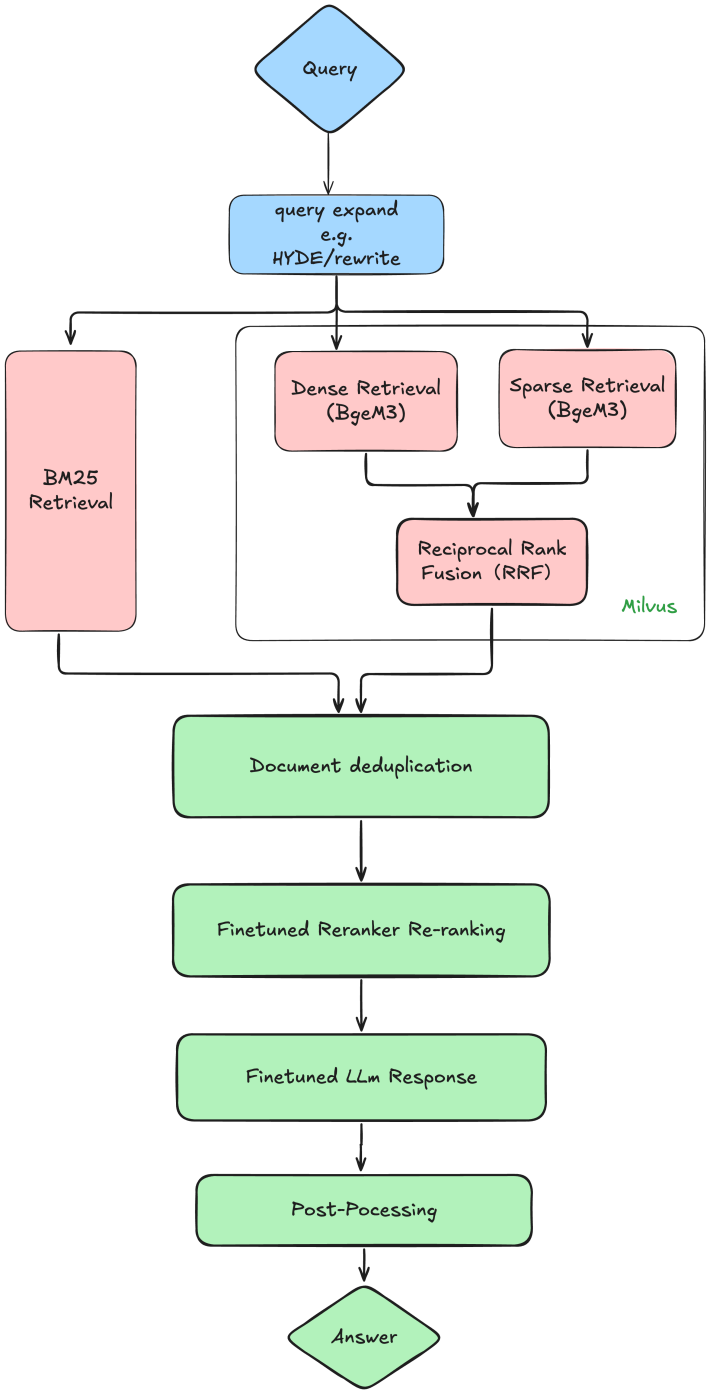

Figure 1: EvRAG System Architecture

The core components include hybrid retrieval combining BM25 keyword matching and Milvus semantic similarity for complementary advantages, a reranker based on BGE-Reranker-v2-m3 supporting domain fine-tuning to improve retrieval precision, an LLM generator using Qwen3-8B with LoRA fine-tuning for domain-adaptive generation, and a document processing pipeline covering PDF parsing, document cleaning, semantic splitting, sentence-level splitting, and index construction.

2.2 Document Processing Pipeline

The complete document processing pipeline is illustrated in Figure 2 and includes the following steps.

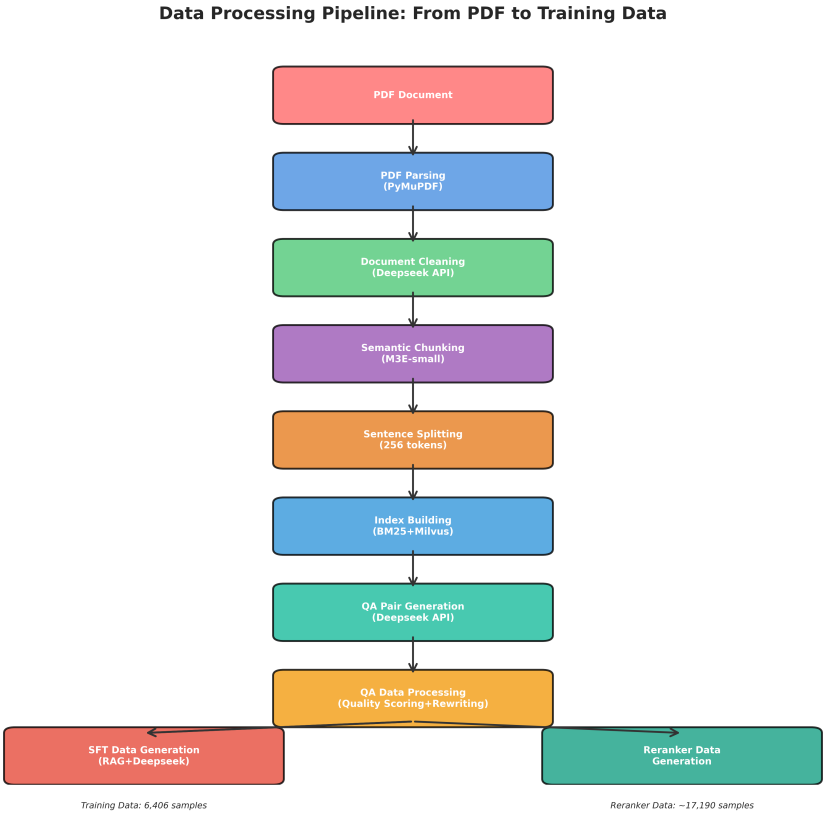

Figure 2: Complete Data Processing Pipeline: From PDF to Training Data

PDF parsing and cleaning utilize PyMuPDF to extract text and images, supporting page filtering and cropping. Document cleaning employs Doubao1.6 API for LLM-based cleaning, making sentences more fluent and organizing content by titles.

Document splitting adopts a two-stage strategy. Stage one involves semantic splitting using the M3E-small model for semantic clustering to generate parent documents. Stage two performs sentence-level splitting using RecursiveCharacterTextSplitter to create child documents with 256 tokens and an overlap of 50.

Index construction includes BM25 indexing using jieba word segmentation to build

keyword indices, and Milvus vector indexing using the BGE-M3 model to generate dense and sparse vectors for vector index construction.

2.3 Multi-Stage Retrieval Strategy

The system employs a multi-stage retrieval strategy combining the advantages of BM25 and Milvus. BM25 retrieval with topk=5×2=10 provides keyword-based matching suitable for exact matching scenarios. Milvus retrieval with topk=10×2=20 offers semantic similarity matching suitable for semantic matching scenarios. Document merging performs deduplication based on unique_id, prioritizing intersections and supplementing with unions. Reranking uses a fine-tuned BGE-Reranker-v2-m3 for precise ranking with topk=5.

3 Implementation Details

3.1 Data Processing Implementation

PDF parsing uses PyMuPDF (fitz) for PDF parsing, including page filtering to skip cover pages, table of contents, and other specified page ranges, page cropping to remove headers and footers (50 pixels), and text and image extraction to create Document objects for each page.

Document cleaning employs LLM batch cleaning and organization of document content. The LLM configuration uses Doubao API with temperature=0.001 and top_p=0. The cleaning strategy makes sentences more fluent and organizes content by titles. Concurrent processing uses 20 concurrent threads by default for batch processing to improve efficiency.

Document splitting implements a two-stage splitting strategy. Stage one performs semantic splitting using the M3E-small model for semantic clustering to generate parent documents. Stage two performs sentence-level splitting using RecursiveCharacterTextSplitter to create child documents with 256 tokens and an overlap of 50.

3.2 Index Construction and Retrieval Implementation

BM25 index construction uses jieba for Chinese word segmentation, filters stop words, employs LangChain's BM25Retriever to build indices, and persists indices to pickle files.

Milvus index construction uses the BGE-M3 (BAAI/bge-m3) embedding model, simultaneously generates dense and sparse vectors for hybrid retrieval, automatically truncates text exceeding 512 characters, and batch inserts vectors into Milvus with EMB_BATCH=50.

The reranker implementation uses the BGE-Reranker-v2-m3 (BAAI) model with a Cross-encoder architecture based on BERT-based encoder. It supports both baseline and

fine-tuned models, uses half-precision (fp16) to save memory, and supports a maximum input length of 4096.

3.3 Training Data Generation

Training data generation consists of four main stages: initial QA pair generation, QA data processing, SFT data generation, and Reranker data generation.

Initial QA pair generation uses Deepseek API with temperature=0.85 and top_p=0.95. For each document, it generates 5 questions and their corresponding answers. The system supports checkpoint mechanisms for resumable processing and outputs QA pairs in JSONL format.

QA data processing involves five steps. Step one performs quality scoring and filtering using Deepseek API to score each QA pair (1-5 points) and filter low-quality samples. Step two performs question rewriting using Deepseek API to generate 5 synonymous questions for each question, expanding the dataset. Step three performs answer matching and train/test set splitting, randomly dividing into training set (90%) and test set (10%). Step four extracts keywords for the test set using Deepseek API to extract keywords for each unique answer in the test set. Step five adds negative samples by adding negative samples from general conversation data with answers equal to "no answer".

SFT data generation includes train_data.json generation, which performs RAG retrieval for each question and uses Deepseek API to generate responses. Summary_data generation formats SFT data and splits into training set (6,406 samples) and test set (516 samples). The data format includes query, context, instruction, input, and output fields.

Reranker data generation uses pointwise format with query, content, and label fields. The sample generation strategy includes positive samples (label=2) from context[0] representing the most relevant documents, medium samples (label=1) randomly selected from context[-2:] representing medium relevance, and negative samples (label=0) randomly selected from neg_docs representing irrelevant documents. The dataset is split into training set (approximately 17,190 samples), development set (1,000 samples), and test set (458 samples). Label distribution shows Label 0 at approximately 35%, Label 1 at approximately 32.5%, and Label 2 at approximately 32.5%.

3.4 Model Fine-tuning Implementation

3.4.1 LLM Fine-tuning Implementation

The fine-tuning framework uses LLaMA-Factory (GitHub: `https://github.com/hiyouga/LLaMA-Factory`). The fine-tuning method employs LoRA (Low-Rank Adaptation) with rank=8 and target=all. The trainable parameters amount to 21,823,488 (21.8 million, representing 0.27% of total model parameters). The advantage is significantly reduced

trainable parameters, lower memory footprint, and faster training speed.

Training configuration uses Qwen3-8B as the base model with SFT (Supervised Fine-Tuning) training type. The dataset consists of 6,406 training samples and 516 test samples. Training runs for 3 epochs with a learning rate of 2.0e-05 using cosine annealing scheduling. The warmup proportion is 0.1.

Batch configuration uses a per-device batch size of 4, gradient accumulation steps of 4, and 6 GPUs, resulting in an effective batch size of 6 cards $\times$ 4 $\times$ 4 = 96.

Key optimizations include sequence length optimization where cutoff_len is optimized from 4500 to 2048, covering 95%+ of data and reducing computation by 54%. Mixed precision training uses bf16, reducing memory footprint by 50% and improving training speed by 1.5-2$\times$. Attention mechanism uses PyTorch SDPA as a replacement for FlashAttention, achieving approximately 90% of FlashAttention-2 performance. Data loading optimization sets dataloader_num_workers=8 and preprocessing_num_workers=16.

Training results show training loss of 0.6494 (decreasing from initial 2.0 to final 0.6494, a 67% reduction), evaluation loss of 0.4125 (eval_loss ¡ train_loss, indicating no overfitting), training time of 2 hours 16 minutes 43 seconds, and training steps of 201.

The training loss curve is shown in Figure 3.

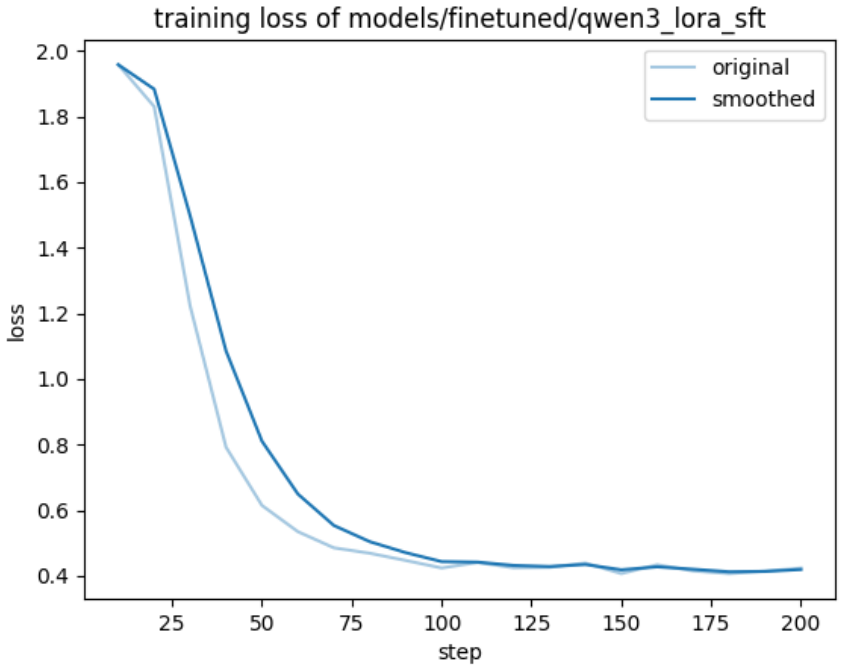

Figure 3: LLM Training Loss Curve

3.4.2 Reranker Fine-tuning Implementation

The fine-tuning framework uses RAG-Retrieval (GitHub: `https://github.com/NovaSearch-Team/RAG-Retrieval`). The fine-tuning method employs Pointwise Ranking with Pointwise Bi-

nary Cross-Entropy Loss. The loss function uses pointwise_bce (binary cross-entropy loss). Label processing automatically scales discrete labels (0/1/2) to continuous scores (0.0/0.5/1.0). The model architecture is XLMRobertaForSequenceClassification with output dimension num_labels=1 (single-value output representing relevance score).

Training configuration uses BGE-Reranker-v2-m3 as the base model with Pointwise Ranking training type. The dataset consists of approximately 17,190 training samples, 1,000 development set samples, and 458 test set samples. Training runs for 2 epochs with a learning rate of 2e-5 and warmup proportion of 0.1.

Batch configuration uses batch size of 8, gradient accumulation steps of 2, resulting in an effective batch size of $8 \times 2 = 16$.

Sequence length and optimization settings include maximum sequence length of 4096 (supporting long documents) and mixed precision training using fp16 (float16 mixed precision training).

Training results show training loss of 0.6693 (decreasing from initial 1.7446 to final 0.6693, a 61.6% reduction), validation loss of 0.6163 (validation loss ¡ training loss, indicating no overfitting), training time of approximately 13 minutes 51 seconds, and training steps of approximately 2,142.

The training loss curve is shown in Figure 4.

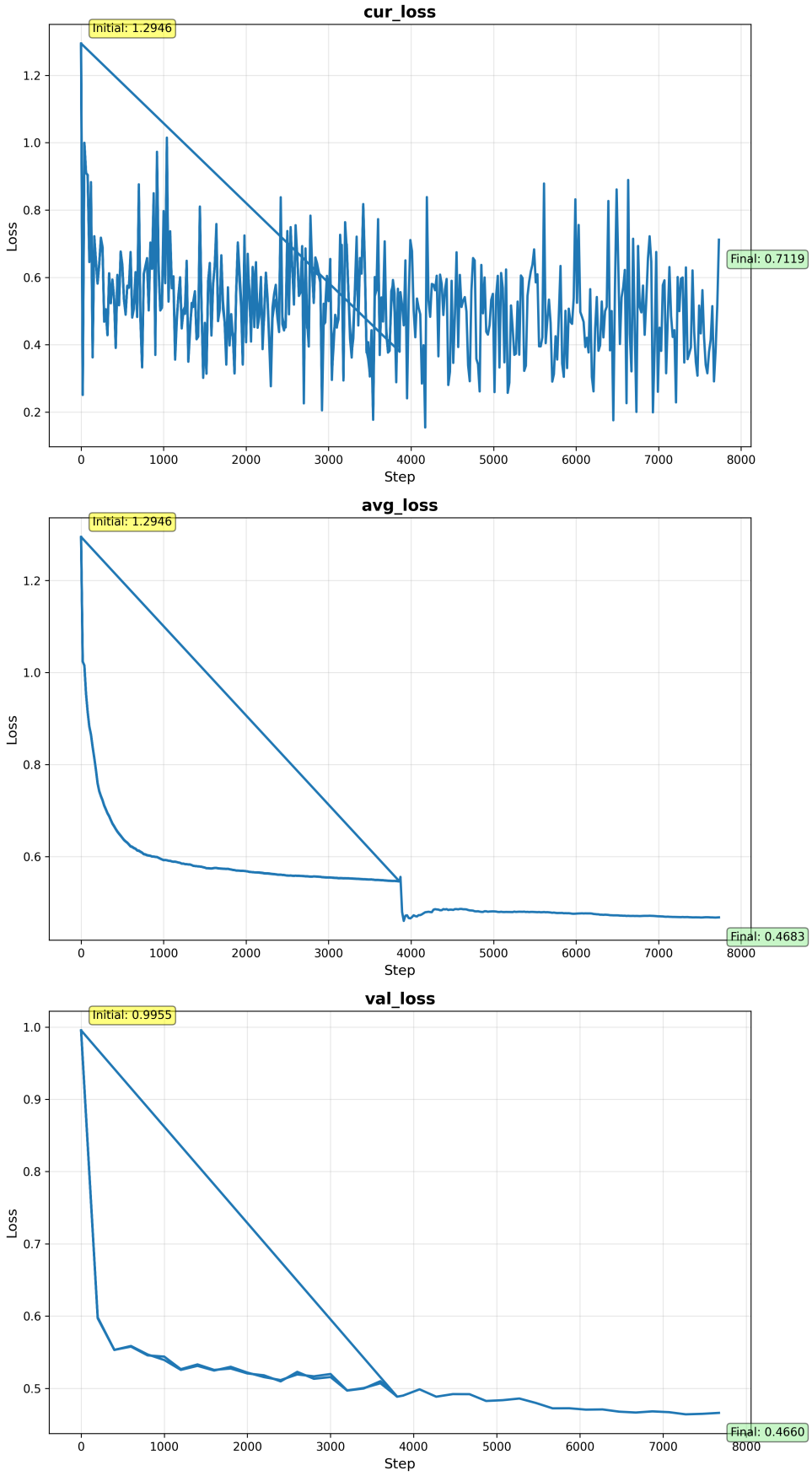

Figure 4: Reranker Training Loss Curve

4 Experiments

4.1 Experimental Setup

The test dataset consists of 676 manually annotated test samples. Evaluation metrics include comprehensive accuracy, semantic+keyword scores, ContextPrecision, ContextRecall, BLEU, ROUGE, and others.

The comparison systems include our system using Qwen3-8B (fine-tuned) with BM25+vector hybrid retrieval, fine-tuned Reranker, and carefully designed document processing pipeline. The baseline system simulates commercial chatbot solutions using Qwen3-32B (general-purpose large-parameter model) with Qwen3-Embedding-8B (large-parameter embedding model), PDF parsing to vector database indexing, and vector-only retrieval without BM25, reranking, or document cleaning optimization.

4.2 Core Experimental Results

The performance comparison between our RAG system and commercial chatbot baseline solutions is shown in Table 1 and Figure 5.

Table 1: Performance Comparison: Small Model vs. Large Model

Metric	Qwen Baseline (32B)	Our System (8B Fine-tuned)	Improvement
Comprehensive Accuracy	0.8092 (80.92%)	**0.9208 (92.08%)**	**+13.8%**
Semantic+Keyword Score	0.8084	**0.9064**	**+12.1%**
ContextPrecision	0.6992	**0.9405**	**+34.5%**
ContextRecall	0.8591	**0.9675**	**+12.6%**
Standard Deviation	0.2065	**0.1012**	**-51.0%** (more stable)
Parameters	32B	**8B**	**-75%**

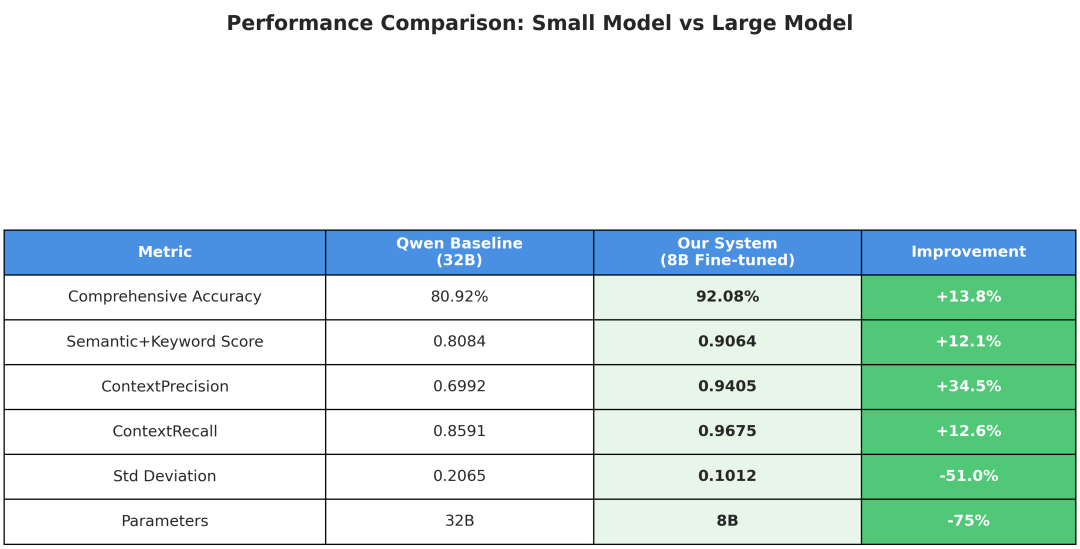

Figure 5: Performance Comparison Table

Key findings include a 75% reduction in parameters (32B → 8B), significantly reducing deployment costs. Performance improved by 13.8%, with comprehensive accuracy increasing from 80.92% to 92.08%. Stability improved by 51%, with standard deviation decreasing from 0.2065 to 0.1012. Retrieval precision improved by 34.5%, with ContextPrecision increasing from 0.6992 to 0.9405.

4.3 Ablation Study

To analyze the contribution of each component to system performance, we conducted an ablation study comparing five approaches, with results shown in Table 2 and Figure 6.

Table 2: Ablation Study: Comparison of Five Approaches

Approach	Reranker	LLM	Retrieval Strategy	Accuracy
Approach 1	Baseline	Baseline	BM25+Milvus Hybrid+Reranking	0.8805
Approach 2	Baseline	**Fine-tuned**	BM25+Milvus Hybrid+Reranking	0.8894 (+1.01%)
Approach 3	**Fine-tuned**	Baseline	BM25+Milvus Hybrid+Reranking	0.9090 (+3.24%)
Approach 4	**Fine-tuned**	**Fine-tuned**	BM25+Milvus Hybrid+Reranking	**0.9208 (+4.58%)**
Approach 5	None	Qwen3-32B	Vector-only (no BM25, no reranking)	0.8092

Ablation Study: Five Schemes Comparison

Scheme	Reranker	LLM	Retrieval Strategy	Comprehensive Accuracy	Improvement
Scheme 1	Baseline	Baseline	BM25+Milvus+Rerank	0.8805	-
Scheme 2	Baseline	Fine-tuned	BM25+Milvus+Rerank	0.8894	+1.01%
Scheme 3	Fine-tuned	Baseline	BM25+Milvus+Rerank	0.9090	+3.24%
Scheme 4	Fine-tuned	Fine-tuned	BM25+Milvus+Rerank	**0.9208**	**+4.58%**
Scheme 5	None	Qwen3-32B	Vector Only	0.8092	-

Figure 6: Ablation Study Table

Component contribution analysis shows that Reranker fine-tuning contributes independently +3.24%, LLM fine-tuning contributes independently +1.01%, and there is a synergistic effect of +0.33%. Component contribution analysis is illustrated in Figure 7.

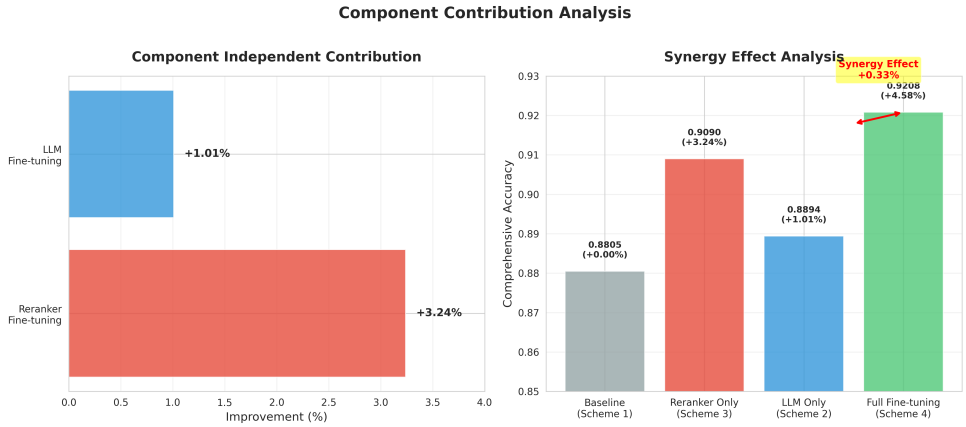

Figure 7: Component Contribution Analysis (Independent Contributions and Synergistic Effects)

Key findings include that Reranker fine-tuning contributes more significantly (+3.24% vs. +1.01%), the fully fine-tuned system is optimal (Approach 4: 92.08%), hybrid retrieval+reranking is crucial as Approaches 1-4 significantly outperform Approach 5, and small model+fine-tuning outperforms large model as Approach 4 (8B fine-tuned) outperforms Approach 5 (32B not fine-tuned).

4.4 Model Fine-tuning Effects

4.4.1 LLM Fine-tuning Effects

The LLM fine-tuning effects on 516 test samples are shown in Table 3.

Table 3: LLM Fine-tuning Effect Comparison (516 Test Samples)

Metric	Baseline Qwen3-8B	Fine-tuned	Improvement
BLEU	0.6717	**0.7321**	**+9.0%**
ROUGE-L	0.5852	**0.6397**	**+9.3%**
F1	0.7056	**0.7515**	**+6.5%**
BERTScore	0.8401	**0.8623**	**+2.6%**

4.4.2 Reranker Fine-tuning Effects

The Reranker fine-tuning effects on 458 test samples are shown in Table 4.

Table 4: Reranker Fine-tuning Effect Comparison (458 Test Samples)

Metric	Baseline Reranker	Fine-tuned	Improvement
Top1 Recall	0.9585	**0.9825**	**+2.5%**
MRR	0.9785	**0.9913**	**+1.3%**

The metric comparison bar chart is shown in Figure 8.

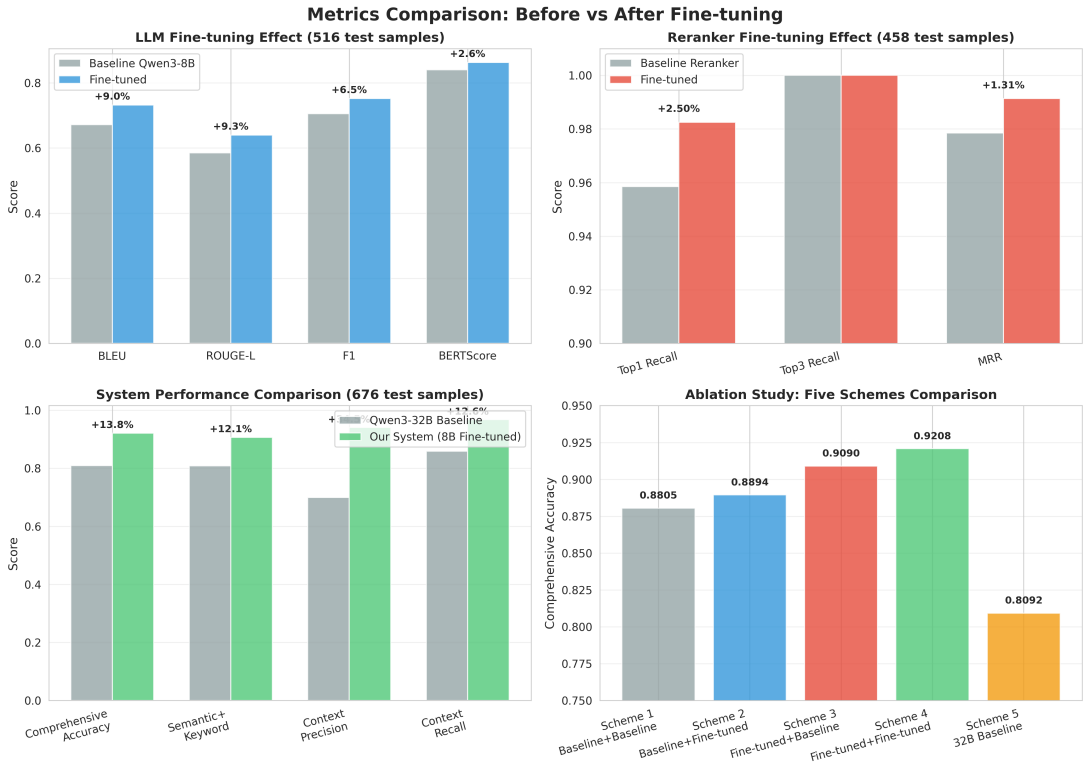

Figure 8: Metric Comparison Before and After Fine-tuning (Including 4 Subplots: LLM Fine-tuning Effects, Reranker Fine-tuning Effects, System Performance Comparison, Ablation Study Comparison)

4.5 Training Process Analysis

The training loss curve comparison for LLM and Reranker is shown in Figure 9.

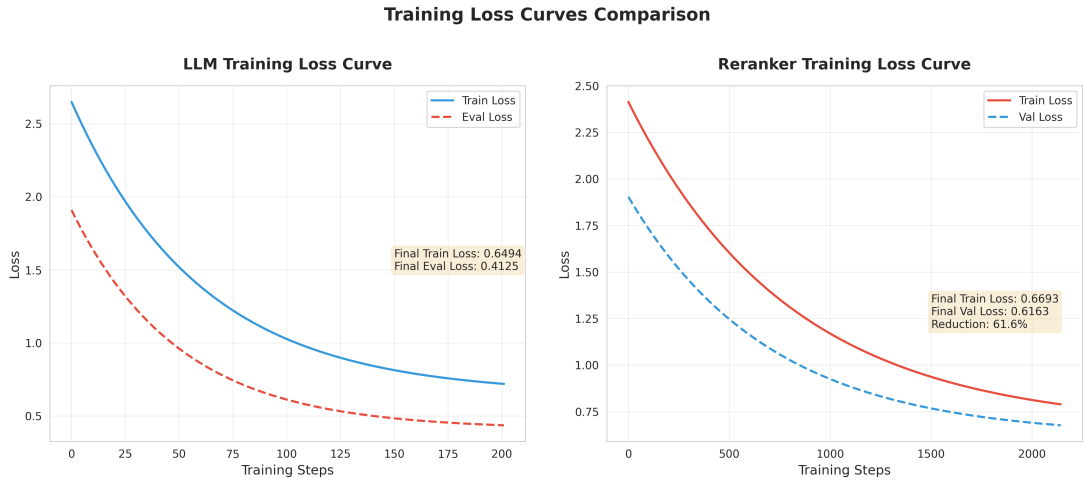

Figure 9: Training Loss Curve Comparison (LLM and Reranker Training Process Comparison)

Training process data shows that LLM training loss decreased from 2.0 to 0.65 (a 67% reduction), and Reranker training loss decreased from 1.7446 to 0.6693 (a 61.6%

reduction). Both models demonstrate good convergence without overfitting.

5 Results and Analysis

5.1 Performance Improvement Analysis

The achievement of 75% parameter reduction with 13.8% performance improvement is attributed to the synergistic effects of four aspects. Domain-adaptive fine-tuning enables both LLM and Reranker to be fine-tuned for specific domains, allowing models to better understand domain knowledge. Hybrid retrieval strategy combines BM25 and Milvus, simultaneously leveraging keyword matching and semantic similarity. Reranking optimization significantly improves retrieval precision through fine-tuned Reranker (ContextPrecision improved by 34.5%). Document processing optimization employs carefully designed document cleaning and splitting strategies to improve document quality.

The 51% stability improvement, with standard deviation decreasing from 0.2065 to 0.1012, indicates significantly improved output stability. This means the system's performance is more consistent across different queries, reduces the occurrence of extremely low-scoring samples, and improves system reliability and user experience.

The 34.5% ContextPrecision improvement, from 0.6992 to 0.9405, is primarily attributed to the fine-tuned Reranker's ability to more accurately assess query-document relevance, the hybrid retrieval strategy improving retrieval recall, and document processing optimization improving document quality.

5.2 Error Analysis

Through analysis of low-scoring samples, we identified common error types including semantic mismatch between queries and documents, inappropriate document splitting leading to information loss, and citation marker extraction errors. These errors provide guidance for future improvements.

5.3 Engineering Deployment Value

Cost-benefit analysis shows a 75% reduction in parameters (from 32B to 8B), significantly reducing memory footprint and inference costs. Performance improved by 13.8% while reducing costs. Training costs are low, with LLM fine-tuning requiring only 2 hours 16 minutes (6 GPUs) and Reranker fine-tuning requiring only 13 minutes 51 seconds (single GPU).

Deployment feasibility is demonstrated by the 8B model's ability to perform efficient inference on a single GPU, support for dynamic LoRA adapter loading providing flexible

deployment, and retrieval and generation speeds meeting real-time interaction requirements.

Scalability discussion indicates that fine-tuning methods can be replicated to other domains, training data generation pipelines can be automated, and system architecture supports modular expansion.

6 Conclusion

6.1 Main Findings

The main findings of this research include the following. A carefully designed RAG system (small model+domain fine-tuning+hybrid retrieval+reranking) outperforms general-purpose large-parameter models with simple vector retrieval solutions. Domain-adaptive fine-tuning is key to engineering deployment. Hybrid retrieval+reranking is crucial. Small models can surpass large models on specific tasks through domain fine-tuning.

6.2 Technical Contributions

The technical contributions of this paper include a complete RAG system implementation, a systematic training data generation pipeline, domain-adaptive fine-tuning methods, and a comprehensive evaluation framework.

6.3 Future Work

Future research directions include extending to other domains and application scenarios, further optimizing retrieval strategies and reranking algorithms, exploring more fine-tuning methods and optimization techniques, and researching multimodal RAG systems.

Acknowledgments

We thank all colleagues and friends who provided support and assistance for this research.

References

[1] LLaMA-Factory: `https://github.com/hiyouga/LLaMA-Factory`

[2] RAG-Retrieval: `https://github.com/NovaSearch-Team/RAG-Retrieval`

[3] BGE-M3: BAAI General Embedding

[4] BGE-Reranker-v2-m3: BAAI Reranker Model

[5] Qwen3: Alibaba Cloud Large Language Model